INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Implementación de un árbol binario en Java
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	3 – "B"
Alumnos participantes:	Jullisa Altamirano
Asignatura:	Estructura de Datos
Docente:	Ing. José Caiza

II. INFORME

1. Actividades por desarrollar

Práctica Implementación de un árbol binario en Java

1. Ejercicio

Caso práctico: sistema de organización jerárquica de departamentos universitarios. La Universidad requiere un programa que represente sus dependencias académicas mediante un árbol binario; cada nodo almacena el nombre de una dependencia y tiene como máximo dos hijos. La aplicación debe construir, visualizar y analizar la estructura, que contiene 9 nodos, 5 hojas y una altura de 3 aristas:



2. Solución propuesta

Se definen tres clases organizadas en paquetes: Nodo<T> (modelo), ArbolBinario<T> (negocio) y Main (app). Nodo<T> almacena un dato y las referencias a sus hijos izquierdo y derecho; ArbolBinario<T> mantiene la referencia a la raíz y ofrece las operaciones de construcción y consulta; Main contiene el menú interactivo con Scanner y los casos de prueba. Las operaciones contarNodos(), contarHojas(), altura() y buscar() son recursivas: un método público invoca a uno privado que recibe el nodo actual, y el caso base es el nodo null (0 nodos, 0 hojas y altura -1). La inserción nunca sobrescribe: si el hijo ya existe, el método devuelve false. Clases:

Clase	Responsabilidad
Nodo<T> (modelo)	Representa un nodo del árbol: guarda dato, izquierdo y derecho; indica si es hoja (esHoja) y cuántos hijos tiene (grado); incluye getters y setters.
ArbolBinario<T> (negocio)	Mantiene la raíz; crea la raíz, agrega hijos izquierdo y derecho, cuenta nodos y hojas, calcula la altura, busca nodos y muestra el árbol en consola.
Main (app)	Construye el árbol inicial, ejecuta el menú de consola con Scanner y los casos de prueba obligatorios.
Scanner	Lectura de la opción del menú y del nombre del nodo a consultar.

3. TDA

Se documentan los TDA/clases principales de cada ejercicio.

Valores:

- raiz: Nodo<T> — referencia al nodo raíz del árbol; es null cuando el árbol está vacío.

- dato: T — información almacenada en un nodo (en la práctica, el nombre de una dependencia).
- izquierdo, derecho: Nodo<T> — referencias a los hijos de un nodo; null cuando no existen.
- opcion: int — opción seleccionada en el menú.
- nombre: String — nombre del nodo consultado por el usuario.

Operaciones:

- crearRaiz(dato) : ArbolBinario x T --> boolean (raiz = new Nodo<>(dato))
- agregarIzquierdo(padre, dato) : ArbolBinario x Nodo x T --> boolean (padre.setIzquierdo(new Nodo<>(dato)) si está libre)
- agregarDerecho(padre, dato) : ArbolBinario x Nodo x T --> boolean (padre.setDerecho(new Nodo<>(dato)) si está libre)
- contarNodos() : ArbolBinario --> int (1 + nodos del subárbol izquierdo + nodos del derecho)
- contarHojas() : ArbolBinario --> int (suma de las hojas de ambos subárboles)
- altura() : ArbolBinario --> int (1 + máximo entre las alturas de los hijos; vacío = -1)
- esVacio() : ArbolBinario --> boolean (raiz == null)
- buscar(dato) : ArbolBinario x T --> Nodo (búsqueda recursiva por dato)
- mostrarArbol() : ArbolBinario --> void (imprime la estructura en consola)
- esHoja() : Nodo --> boolean (izquierdo == null && derecho == null)
- grado() : Nodo --> int (número de hijos: 0, 1 o 2)

4. Relación entre requisitos y codigo

Requisito	**Aplicación**
Clase genérica Nodo<T> con dato, izquierdo y derecho	Clase Nodo<T> del paquete modelo, con sus tres atributos privados, constructor, getters y setters.
esHoja() y grado()	esHoja() devuelve izquierdo == null && derecho == null; grado() suma 1 por cada hijo no nulo.
crearRaiz, agregarIzquierdo y agregarDerecho	Métodos de ArbolBinario que devuelven boolean; rechazan la inserción si el padre es null o la posición ya está ocupada.
contarNodos(), contarHojas() y altura() recursivos	Métodos públicos que delegan en métodos privados recursivos con caso base nodo == null.
Altura en aristas (vacío = -1, un nodo = 0)	altura(null) devuelve -1; una hoja resulta 1 + max(-1, -1) = 0.
esVacio() y mostrarArbol()	esVacio() compara la raíz con null; mostrarArbol() imprime el árbol por niveles con sangría ([I] izquierdo, [D] derecho).
Menú interactivo con Scanner	Main: ciclo do-while con switch de las opciones 1 a 9 y lectura validada con Scanner.
Consultar grado y verificar hoja de un nodo	Opciones 6 y 7 del menú: se busca el nodo por nombre con buscar() y se invocan grado() y esHoja().
Casos de prueba obligatorios	Opción 8 del menú: árbol vacío, un solo nodo, árbol inicial, inserción en posición ocupada y grado de FISEI y SOFTWARE.
No usar TreeSet, TreeMap, PriorityQueue, NavigableSet ni NavigableMap	La estructura se construye solo con la clase Nodo<T> y referencias propias.

5. Pasos de construcción

Paso 1 – Atributos de Nodo<T>

```
public class Nodo<T> {
    private T dato;
    private Nodo<T> izquierdo;
    private Nodo<T> derecho;

    public Nodo(T dato) {
        this.dato = dato;
    }
}
```

Paso 2 – esHoja() y grado()

```
public boolean esHoja() {
    return izquierdo == null && derecho == null;
}

public int grado() {
    int grado = 0;
    if (izquierdo != null) grado++;
    if (derecho != null) grado++;
    return grado;
}
```

Paso 3 – Inserción de hijos

```
public boolean agregarIzquierdo(Nodo<T> padre, T dato) {
    if (padre == null || padre.getIzquierdo() != null) {
        return false;   // posición ocupada
    }
    padre.setIzquierdo(new Nodo<>(dato));
    return true;
}
```

Paso 4 – contarNodos() recursivo

```
private int contarNodos(Nodo<T> nodo) {
    if (nodo == null) {
        return 0;
    }
    return 1 + contarNodos(nodo.getIzquierdo())
             + contarNodos(nodo.getDerecho());
}
```

Paso 5 – contarHojas() recursivo

```
private int contarHojas(Nodo<T> nodo) {
    if (nodo == null) {
        return 0;
    }
    if (nodo.esHoja()) {
        return 1;
    }
    return contarHojas(nodo.getIzquierdo())
         + contarHojas(nodo.getDerecho());
}
```

Paso 6 – altura() recursiva (en aristas)

```
private int altura(Nodo<T> nodo) {
    if (nodo == null) {
        return -1;
    }
    return 1 + Math.max(altura(nodo.getIzquierdo()),
                        altura(nodo.getDerecho()));
}
```

Paso 7 – Construcción del árbol inicial (Main)

```
ArbolBinario<String> arbol = new ArbolBinario<>();
arbol.crearRaiz("UNIVERSIDAD");
Nodo<String> universidad = arbol.getRaiz();

arbol.agregarIzquierdo(universidad, "FISEI");
arbol.agregarDerecho(universidad, "DIDE");

Nodo<String> fisei = universidad.getIzquierdo();
Nodo<String> dide = universidad.getDerecho();
arbol.agregarIzquierdo(fisei, "SOFTWARE");
arbol.agregarDerecho(fisei, "TI");
arbol.agregarIzquierdo(dide, "INVEST.");
arbol.agregarDerecho(dide, "VINCULACIÓN");

Nodo<String> software = fisei.getIzquierdo();
arbol.agregarIzquierdo(software, "NIVEL1");
arbol.agregarDerecho(software, "NIVEL3");
```

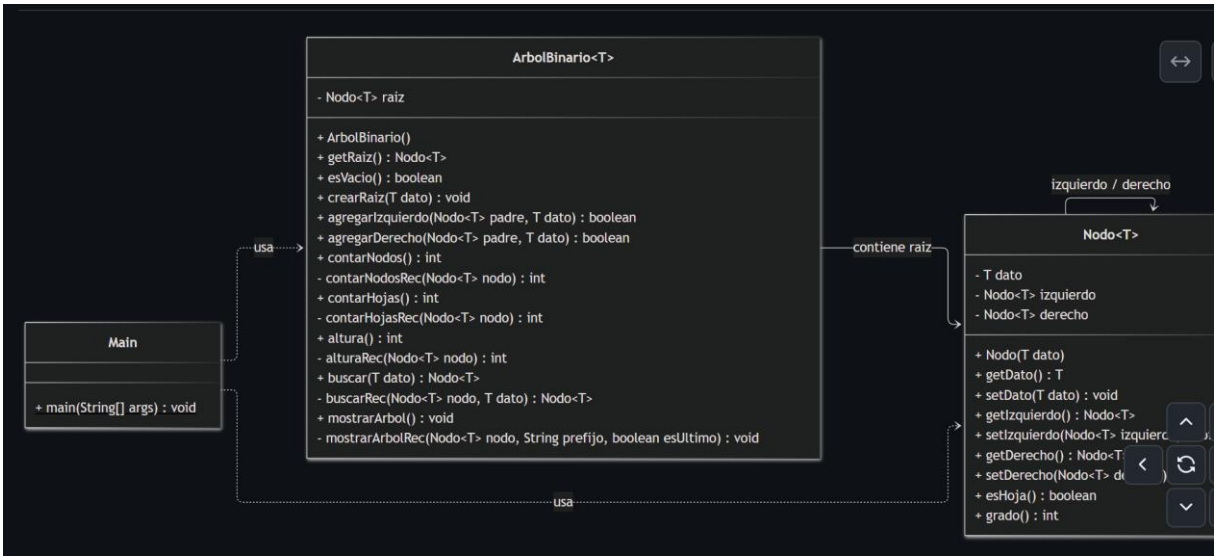
Paso 8 – Menú interactivo con Scanner

```
Scanner sc = new Scanner(System.in);
int opcion;
do {
    mostrarMenu();
    opcion = leerEntero("Seleccione una opción: ");
    switch (opcion) {
        case 1: arbol = crearArbolInicial(); break;
        case 2: arbol.mostrarArbol(); break;
        case 3: System.out.println("Nodos: " + arbol.contarNodos()); break;
        case 4: System.out.println("Hojas: " + arbol.contarHojas()); break;
        case 5: System.out.println("Altura: " + arbol.altura()); break;
        case 6: consultarGrado(arbol); break;
        case 7: verificarHoja(arbol); break;
        case 8: ejecutarCasosDePrueba(); break;
        case 9: System.out.println("Saliendo del programa..."); break;
        default: System.out.println("Opción no válida.");
    }
} while (opcion != 9);
```

6. Tipos de datos utilizados

Tipo	Categoria	Variables
Nodo<T>	Referencia (clase genérica)	raiz, izquierdo, derecho, padre, universidad, fisei, dide, software
ArbolBinario<String>	Referencia (clase genérica)	arbol, vacio, unico, inicial
T / String	Referencia	dato de cada nodo; nombre del nodo consultado
int	Primitivo	opcion, grado, resultados de contarNodos(), contarHojas() y altura()
boolean	Primitivo	resultado de esHoja(), esVacio() y de las inserciones (agregado)
Scanner	Referencia	sc

7. Diagrama de clases



8. Especificación del TDA

Estado: el árbol mantiene una única referencia a la raíz; cada nodo guarda un dato y las referencias a sus hijos izquierdo y derecho (máximo dos). El árbol vacío tiene raiz = null. No comparte estado con ninguna otra estructura.

Metodo	**Resultado**
crearRaiz(dato)	Si el árbol no tiene raíz, crea el nodo raíz con el dato y devuelve true; si ya existe, devuelve false.
agregarIzquierdo(padre, dato)	Si el padre existe y no tiene hijo izquierdo, lo crea y devuelve true; si la posición está ocupada, no modifica el árbol y devuelve false.
agregarDerecho(padre, dato)	Igual que el anterior, pero sobre el hijo derecho.
contarNodos()	Devuelve el total de nodos; 0 si el árbol está vacío.
contarHojas()	Devuelve cuántos nodos no tienen hijos; 0 si el árbol está vacío.
altura()	Devuelve la longitud, en aristas, del camino más largo desde la raíz hasta una hoja; -1 si el árbol está vacío.
esVacio()	Devuelve true si la raíz es null.
buscar(dato)	Devuelve el primer nodo cuyo dato coincide (sin distinguir mayúsculas/minúsculas) o null si no existe.
mostrarArbol()	Imprime en consola la estructura jerárquica, indicando [I] izquierdo y [D] derecho.

Invariantes:

- Cada nodo tiene como máximo dos hijos: uno izquierdo y uno derecho.
- La raíz es el único nodo sin padre; todos los demás nodos tienen exactamente un padre.
- Una hoja es un nodo con grado 0, y su altura es siempre 0.
- La altura del árbol vacío es -1 y la de un árbol con un solo nodo es 0.
- Una inserción nunca sobrescribe un hijo existente: si la posición está ocupada, el árbol queda igual.

9. Pseudocodigo

```
INICIO
arbol <- arbol vacio (raiz <- nulo)
REPITE
```

```
    mostrar menu
    leer opcion
    SEGUN opcion HACER
        1: arbol <- crear arbol inicial (9 nodos)
        2: arbol.mostrarArbol()
        3: imprimir arbol.contarNodos()
        4: imprimir arbol.contarHojas()
        5: imprimir arbol.altura()
        6: leer nombre
           nodo <- arbol.buscar(nombre)
           SI nodo EXISTE ENTONCES imprimir nodo.grado()
           SINO imprimir "El nodo no existe"
        7: leer nombre
           nodo <- arbol.buscar(nombre)
           SI nodo EXISTE ENTONCES imprimir nodo.esHoja()
           SINO imprimir "El nodo no existe"
        8: ejecutar casos de prueba
        9: imprimir mensaje de salida
    FIN SEGUN
HASTA opcion = 9
FIN

FUNCION altura(nodo)
    SI nodo = nulo ENTONCES RETORNAR -1
    RETORNAR 1 + MAXIMO(altura(nodo.izquierdo), altura(nodo.derecho))
FIN FUNCION

FUNCION contarNodos(nodo)
    SI nodo = nulo ENTONCES RETORNAR 0
    RETORNAR 1 + contarNodos(nodo.izquierdo) + contarNodos(nodo.derecho)
FIN FUNCION

FUNCION contarHojas(nodo)
    SI nodo = nulo ENTONCES RETORNAR 0
    SI nodo.izquierdo = nulo Y nodo.derecho = nulo ENTONCES RETORNAR 1
    RETORNAR contarHojas(nodo.izquierdo) + contarHojas(nodo.derecho)
FIN FUNCION
```

10. Prueba de escritorio

Construcción del árbol inicial paso a paso, con el estado del árbol después de cada operación:

Paso	Operación	Nodos	Hojas	Altura
0	Árbol vacío	0	0	-1
1	crearRaiz("UNIVERSIDAD")	1	1	0
2	agregarIzquierdo(UNIVERSIDAD, "FISEI")	2	1	1
3	agregarDerecho(UNIVERSIDAD, "DIDE")	3	2	1
4	agregarIzquierdo(FISEI, "SOFTWARE")	4	2	2
5	agregarDerecho(FISEI, "TI")	5	3	2
6	agregarIzquierdo(DIDE, "INVEST.")	6	3	2
7	agregarDerecho(DIDE, "VINCULACIÓN")	7	4	2
8	agregarIzquierdo(SOFTWARE, "NIVEL1")	8	4	3
9	agregarDerecho(SOFTWARE, "NIVEL3")	9	5	3
10	agregarIzquierdo(FISEI, "NUEVO") – posición ocupada (devuelve false)	9	5	3
11	grado(FISEI) = 2 y grado(SOFTWARE) = 2 (consulta, sin cambios)	9	5	3

Cálculo recursivo de altura() sobre el árbol inicial (de las hojas hacia la raíz):

Llamada	**Cálculo**	**Resultado**
altura(NIVEL1), altura(NIVEL3), altura(TI)	1 + max(-1, -1)	0
altura(INVEST.), altura(VINCULACIÓN)	1 + max(-1, -1)	0
altura(SOFTWARE)	1 + max(0, 0)	1
altura(FISEI)	1 + max(altura(SOFTWARE) = 1, altura(TI) = 0)	2
altura(DIDE)	1 + max(0, 0)	1
altura(UNIVERSIDAD)	1 + max(altura(FISEI) = 2, altura(DIDE) = 1)	3

2. Resultados obtenidos

El árbol inicial se construyó correctamente con 9 nodos, 5 hojas (TI, NIVEL1, NIVEL3, INVEST. y VINCULACIÓN) y una altura de 3 aristas, valores que coinciden con la prueba de escritorio.

Las operaciones de conteo y altura, implementadas con recursividad, respondieron bien en los casos límite: el árbol vacío devolvió 0 nodos y altura -1, y el árbol con un solo nodo devolvió 1 nodo, 1 hoja y altura 0.

El intento de insertar un hijo en una posición ocupada (FISEI ya tiene hijo izquierdo) fue rechazado: el método devolvió false y el árbol no se modificó. Las consultas de grado dieron 2 para FISEI y 2 para SOFTWARE.

El menú interactivo con Scanner permitió crear, visualizar y consultar el árbol; además valida la entrada y avisa cuando el nodo consultado no existe o cuando el árbol aún no ha sido creado.

3. Preguntas

1. ¿Por qué la altura de un árbol binario depende del camino más largo desde la raíz hasta una hoja?

Porque la altura mide cuántos niveles hay que descender, como mínimo, para alcanzar el nodo más profundo. Un árbol tiene varias ramas de distinta longitud, pero la estructura llega tan abajo como su rama más larga; las ramas cortas no aumentan ni reducen ese valor. Por eso, en el código, la altura de un nodo es 1 + el máximo entre la altura de su hijo izquierdo y la de su hijo derecho, y se toma -1 para un subárbol vacío, de modo que una hoja resulta con altura 0. En el árbol de la práctica, la rama UNIVERSIDAD → FISEI → SOFTWARE → NIVEL1 tiene 3 aristas y es la más larga, mientras que UNIVERSIDAD → DIDE → INVEST. solo tiene 2; por eso la altura del árbol es 3.

2. ¿Qué diferencias existen entre almacenar información en una lista enlazada y en un árbol binario?

En una lista enlazada cada nodo tiene un solo sucesor, así que los datos forman una secuencia lineal: para llegar al último elemento hay que recorrer todos los anteriores y buscar cuesta O(n). En un árbol binario cada nodo puede tener hasta dos hijos, lo que permite representar relaciones jerárquicas (como universidad → facultad → carrera) y, si el árbol está ordenado y equilibrado, buscar en O(log n), porque en cada paso se descarta una rama. La contrapartida es que el árbol es más complejo de implementar (recorridos recursivos, dos referencias por nodo) y, si queda desbalanceado, puede degenerar en una lista con el mismo costo lineal. La lista conviene cuando importa el orden de llegada; el árbol, cuando importan la jerarquía o la búsqueda eficiente.

3. ¿Cómo podría utilizarse un árbol binario en el proyecto integrador SmartETL para organizar o buscar registros?

En SmartETL, que procesa registros de distintas fuentes, un árbol binario de búsqueda podría servir como índice en memoria: cada nodo guardaría una clave (por ejemplo, el identificador o la fecha del registro) y una referencia al registro, de modo que localizar un registro, detectar duplicados durante la

transformación o consultar rangos sería más rápido que recorrer toda la lista. También podría representar la jerarquía de las etapas del proceso (extracción, limpieza, transformación y carga) o clasificar registros mediante reglas de decisión de tipo sí/no, donde cada nodo es una condición y las hojas son el destino final del registro.

4. Anexos

Actualizaciones o mejoras del Software

Github: https://github.com/jullisaaltamirano2017-boop/Implementacion_de_un_arbol_Binario

Capturas de ejecución:

```
--- SISTEMA DE ORGANIZACIÓN JERÁRQUICA - ÁRBOL BINARIO ---
1. Crear el árbol inicial (Universidad)
2. Visualizar el árbol
3. Consultar el número total de nodos
4. Consultar el número total de hojas
5. Consultar la altura del árbol
6. Consultar el grado de un nodo
7. Verificar si un nodo es hoja
8. Probar casos de acuerdo a requerimientos
9. Salir
Seleccione una opción: 2
```

```
Estructura del Árbol:
UNIVERSIDAD
+-- [I] FISEI
|   +-- [I] SOFTWARE
|   |   +-- [I] NIVEL1
|   |   +-- [D] NIVEL3
|   +-- [D] TI
+-- [D] DIDE
    +-- [I] INVEST.
    +-- [D] VINCULACIÓN
```

```
 --- SISTEMA DE ORGANIZACIÓN JERÁRQUICA - ÁRBOL BINARIO ---
 1. Crear el árbol inicial (Universidad)
 2. Visualizar el árbol
 3. Consultar el número total de nodos
 4. Consultar el número total de hojas
 5. Consultar la altura del árbol
 6. Consultar el grado de un nodo
 7. Verificar si un nodo es hoja
 8. Probar casos de acuerdo a requerimientos
 9. Salir
 Seleccione una opción: 3
 Número total de nodos: 9

--- SISTEMA DE ORGANIZACIÓN JERÁRQUICA - ÁRBOL BINARIO ---
1. Crear el árbol inicial (Universidad)
2. Visualizar el árbol
3. Consultar el número total de nodos
4. Consultar el número total de hojas
5. Consultar la altura del árbol
6. Consultar el grado de un nodo
7. Verificar si un nodo es hoja
8. Probar casos de acuerdo a requerimientos
9. Salir
Seleccione una opción: 4
Número total de hojas: 5

 --- SISTEMA DE ORGANIZACIÓN JERÁRQUICA - ÁRBOL BINARIO ---
 1. Crear el árbol inicial (Universidad)
 2. Visualizar el árbol
 3. Consultar el número total de nodos
 4. Consultar el número total de hojas
 5. Consultar la altura del árbol
 6. Consultar el grado de un nodo
 7. Verificar si un nodo es hoja
 8. Probar casos de acuerdo a requerimientos
 9. Salir
 Seleccione una opción: 5
 Altura del árbol (en aristas): 3
```

```
--- SISTEMA DE ORGANIZACIÓN JERÁRQUICA - ÁRBOL BINARIO ---
1. Crear el árbol inicial (Universidad)
2. Visualizar el árbol
3. Consultar el número total de nodos
4. Consultar el número total de hojas
5. Consultar la altura del árbol
6. Consultar el grado de un nodo
7. Verificar si un nodo es hoja
8. Probar casos de acuerdo a requerimientos
9. Salir
Seleccione una opción: 6
Ingrese el nombre del nodo a consultar (ej. FISEI, SOFTWARE): FISEI
El grado del nodo [FISEI] es: 2

--- SISTEMA DE ORGANIZACIÓN JERÁRQUICA - ÁRBOL BINARIO ---
1. Crear el árbol inicial (Universidad)
2. Visualizar el árbol
3. Consultar el número total de nodos
4. Consultar el número total de hojas
5. Consultar la altura del árbol
6. Consultar el grado de un nodo
7. Verificar si un nodo es hoja
8. Probar casos de acuerdo a requerimientos
9. Salir
Seleccione una opción: 7
Ingrese el nombre del nodo a verificar (ej. NIVEL1, DIDE): NIVEL1
El nodo [NIVEL1] SI es una hoja.
```

```
--- SISTEMA DE ORGANIZACIÓN JERÁRQUICA - ÁRBOL BINARIO ---
1. Crear el árbol inicial (Universidad)
2. Visualizar el árbol
3. Consultar el número total de nodos
4. Consultar el número total de hojas
5. Consultar la altura del árbol
6. Consultar el grado de un nodo
7. Verificar si un nodo es hoja
8. Probar casos de acuerdo a requerimientos
9. Salir
Seleccione una opción: 8

--- DEMOSTRACIÓN DE CASOS DE PRUEBA OBLIGATORIOS ---
-> Árbol Vacío | Nodos: 0 | Altura: -1
-> Árbol Un Nodo | Hojas: 1 | Altura: 0
-> Árbol Inicial | Nodos: 9 | Hojas: 5 | Altura: 3
-> Probando inserción en posición ocupada (FISEI):
-> Grado de FISEI: 2
-> Grado de SOFTWARE: 2
```